

Univerzitet u Sarajevu
Elektrotehnički fakultet
Sarajevo

UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET

P16: BVH RAY TRACER

Izvještaj o naučnom radu

RAD [1]

Parallel Axis Split Tasks for Bounding Volume Construction with OpenMP

Waldemarson i Doggett (2025)

Predmet: Paralelni računarski sistemi
Mentor: Tarik Hrnjić, MA

Autori

Dino Burić - 2502/2026
Ali Fazlić - 2590/2026
Zijad Doglod - 2576/2026
Ajdin Dželo - 2527/19540

Sarajevo, novembar 2026.

Sažetak

Waldemarson i Doggett istražuju koliko se zahtjevan CPU algoritam izgradnje BVH-a može ubrzati dodavanjem malog broja OpenMP direktiva u postojeću sekvencijalnu implementaciju. Algoritam Spatial Split BVH (SBVH) paralelizuju na dva nivoa: rekurzivnim zadacima za podstabla i novim zadacima za pretragu svake ose unutar čvora. Najbolje varijante grade stablo 3 do 5 puta brže, uz zanemariv gubitak kvaliteta. Za P16 rad nudi direktan obrazac paralelne izgradnje BVH-a i listu konkretnih OpenMP zamki koje trebamo uzeti u obzir pri mjerenju.

1. Rad ukratko

Referenca	Waldemarson i Doggett, GRAPP/VISIGRAPP 2025 [1]
Problem	SBVH izgradnja je najsporiji široko korišten način izgradnje BVH-a i loše se skalira tokom prvih podjela.
Tehnika	OpenMP zadaci za rekurzivna podstabla i za pretragu objektnih i prostornih podjela po osama.
Hardver	Izgradnja: Intel Core i7-6700K, te Xeon w7-3465X za veći broj niti. Evaluacija: OpenCL renderer ambijentalne okluzije na NVIDIA RTX 3060.
Glavni rezultat	Do 5 puta brža izgradnja, 3–5 puta za dobre varijante, uz zanemariv pad kvaliteta BVH-a.
Vrijednost za P16	**Visoka.** Koristi isti programski model kao naša V3 i obrađuje dio koji još nismo paralelizovali.

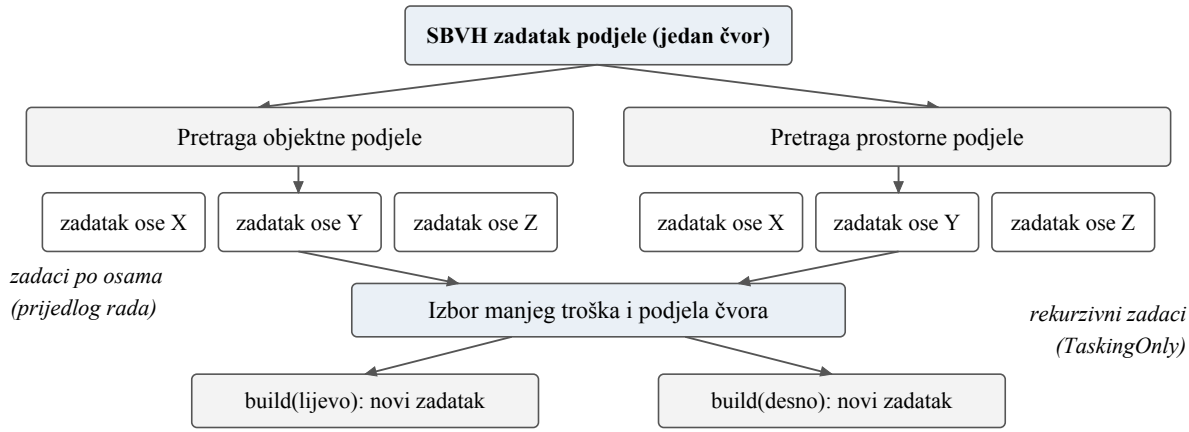
2. Problem koji rad rješava

Izgradnja odozgo prema dolje ima urođen problem skaliranja. Prva podjela obrađuje sve primitive na jednoj niti, naredni nivo ima samo dva zadatka i tako dalje. Sve niti postaju zauzete tek kada nastane dovoljno podstabala, pa se najskuplji koraci izvršavaju uz najmanje paralelizma. Raniji radovi izvlače paralelizam iz ovih prvih podjela dijeljenjem računanja preko svih primitiva, ali to zahtijeva složenu sinhronizaciju [1].

Autori biraju namjerno jednostavniji put: paralelizaciju samo preko tri ose podjele. Time se odriču dijela mogućeg paralelizma, a zauzvrat dobijaju jednostavnu sinhronizaciju i malu izmjenu koda. Ideju primjenjuju na SBVH, koji daje vrlo brza stabla, ali je obično najsporiji za izgradnju jer tokom nje može dijeliti reference na trouglove i stvarati nove.

3. Korištena tehnika

Za svaki čvor SBVH procjenjuje objektnu i prostornu podjelu, izvodi jeftiniju i rekurzivno nastavlja u oba djeteta. Rad označava dvije vrste paralelnih regija (Slika 1): dva rekurzivna poziva izgradnje i petlje po osama X, Y i Z unutar svake pretrage. Druga vrsta je doprinos rada, jer stvara koristan posao dok postoji malo podstabala. Ona bi mogla učiniti isplativim i skuplje heuristike, na primjer veći broj binova kod binned SAH-a.



Slika 1. Dva nivoa OpenMP zadataka u jednoj SBVH podjeli (naš crtež, prema [1], slika 4).

Autori ubacuju OpenMP direktive u osnovnu implementaciju kao izmjene izvornog koda, što omogućava direktno poređenje varijanti iz Tabele 1.

Varijanta	**Rekurzivni pozivi**	**Pretrage po osama**
NoOpenMP	Sekvencijalno	Sekvencijalno
TaskingOnly	omp task	Sekvencijalno
Tasks	omp task	omp task + taskwait
Taskloop	omp task	omp taskloop
ParallelFor	omp task	Ugniježđeni omp parallel for
NoTaskingFor	Sekvencijalno	omp parallel for
NoTaskingTasks	Sekvencijalno	omp task
NoTaskingTaskloop	Sekvencijalno	omp taskloop

Tabela 1. OpenMP varijante evaluirane u [1].

4. Rezultati

Eksperimenti koriste šest scena, od Sponze (262.267 trouglova) do Powerplanta (12.759.246 trouglova), kompajliranih i GCC-om 11.4 i Clangom 14. Renderovanje na zasebnom GPU ray traceru služi samo za procjenu kvaliteta dobivenih BVH stabala. Nalazi su uglavnom prikazani grafikonima, pa ih u Tabeli 2 sažimamo kvalitativno i ne navodimo vrijednosti po scenama.

Nalaz	**Dokaz u [1]**	**Značenje za nas**
Ubrzanje izgradnje	3–5 puta na postavci autora s 8 niti; do 5 puta u odnosu na jednu nit.	Vrijedan dobitak za vrlo malo koda.
Granica skaliranja	Svaka dodatna nit donosi manje. Na Xeonu skaliranje skoro staje između 10 i 20 niti.	Prikazati efikasnost i ne očekivati linearno ubrzanje.
Task ili taskloop	Nema značajne razlike u performansama; taskloop je čitljiviji i ne traži eksplicitnu sinhronizaciju.	Birati prema čitljivosti.
Varijante samo po osama	Umjeren dobitak i nema skaliranja preko 3 niti, jer postoje samo 3 ose. Dodatne niti unose trošak.	Ako se koriste same, ograničiti ih na 3 niti.
Determinizam	Kritična sekcija dodjeljuje indekse čvorova, pa redoslijed čvorova varira. Fragmentacija blago povećava vrijeme renderovanja i njegovu varijaciju.	Paralelna izgradnja može usporiti renderovanje.

Nalaz	Dokaz u [1]	Značenje za nas
Ugniježđeni paralelizam	Radi s Clangom. S GCC-om je ParallelFor višestruko sporiji od osnovne verzije.	Izbjegavati ugniježđene regije.
Trošak na jednoj niti	Sve OpenMP varijante imaju malu, ali primjetnu kaznu kada rade na jednoj niti.	Kao osnovu koristiti program bez OpenMP-a.

Tabela 2. Glavni nalazi rada [1] i njihove posljedice za P16.

5. Kritički osvrt

Rezultat je uvjerljiv kao inženjerski rezultat: pokazuje velik dobitak uz minimalne izmjene, a kvalitet stabla ocjenjuje stvarnim vremenima renderovanja. Ipak, treba imati na umu neka ograničenja. Vrijednost od 3–5 puta odnosi se na SBVH, čija skupa pretraga podjela daje mnogo posla za paralelizaciju. Jednostavniji algoritam izgradnje može imati drugačiji dobitak. Autori prag za male zadatke predlažu tek kao budući rad, pa trošak vrlo malih zadataka u njihovoj implementaciji ostaje otvoreno pitanje.

Potrebna je i jedna ispravka u čitanju hardvera: rad govori o „8-core“ mašini, a naš Checkpoint 1 ponavlja formulaciju o osmojezgrenom procesoru [CP1]. Intel specificira i7-6700K kao procesor s 4 fizičke jezgre i 8 hardverskih niti, pa rezultat treba čitati kao mjerenje na 8 hardverskih niti. U našim mjerenjima zato trebamo odvojeno navoditi fizičke jezgre i hardverske niti.

6. Šta možemo iskoristiti u P16

Naš plan paralelizuje renderovanje u V3, a izgradnju BVH-a ostavlja sekvencijalnom. Rad pokazuje kako zatvoriti tu prazninu, a glavna pouka je da dvije faze zahtijevaju odvojene eksperimente. Sam SBVH nam nije potreban: prostorne podjele donose dupliranje referenci i složenost koje naš projekat ne traži. Isti obrazac zadataka možemo primijeniti na naš jednostavniji algoritam izgradnje.

Ideja iz [1]	Kako je primjenjujemo	Prioritet
parallel for po redovima slike ([1], slika 2)	Osnova V3. Dodajemo poređenje raspoređivanja jer se trošak piksela razlikuje.	Osnovno
Rekurzivni zadaci izgradnje	Nova varijanta V3b: paralelno renderovanje i paralelna izgradnja.	Preporučeno
Prag za zadatke	Zadatak stvarati samo za opsege iznad praga; izmjeriti nekoliko pragova.	Preporučeno
Zadaci po osama	Ako dodamo binned SAH, isprobati *taskloop* preko tri ose za velike čvorove.	Opcionalno
Deterministički raspored čvorova	Svakom podstablu dati fiksan opseg indeksa umjesto zajedničkog brojača.	Preporučeno
Poređenje kompajlera	Isti eksperiment pokrenuti s GCC-om i Clangom.	Opcionalno

*Tabela 3. Plan integracije. Prioritet: **Osnovno** = potrebno za V1–V3; **Preporučeno** = vrijedna nadogradnja; **Opcionalno** = samo ako vrijeme dozvoli.*

```
void build(int node, int first, int last) {          // podstablo posjeduje node .. node + 2*(last-first) - 2
    int count = last - first;
    if (count <= LEAF_SIZE) { makeLeaf(node, first, last); return; }
    int mid = partitionPrimitives(node, first, last);  // prvo medijana, kasnije SAH
    int left = node + 1;
    int right = left + 2 * (mid - first) - 1;       // deterministički indeks, bez zajedničkog brojača
    #pragma omp task if(count > TASK_CUTOFF)
    build(left, first, mid);
    #pragma omp task if(count > TASK_CUTOFF)
    build(right, mid, last);
    #pragma omp taskwait
    updateBounds(node, left, right);
}
// Ulazna tačka: #pragma omp parallel  +  #pragma omp single  oko poziva build(0, 0, n);
```

Listing 1. Predloženi algoritam izgradnje za P16 (naš kod, prema obrascu zadataka iz [1]).

Listing 1 unaprijed alocira 2n−1 čvorova i svakom podstablu daje vlastiti opseg indeksa. Za razliku od kritične sekcije u [1], ovo ne traži sinhronizaciju i daje isti raspored za svaki broj niti, čime se uklanja opisani nedeterminizam i pojednostavljuju provjere ispravnosti. Kratki opsezi mogu ostaviti neiskorištena mjesta, koja možemo sabiti nakon izgradnje.

Eksperimenti direktno slijede iz Tabele 2. Vrijeme izgradnje i renderovanja mjerićemo odvojeno za 1, 2, 4 i dalje do maksimalnog broja niti, a ubrzanje računati u odnosu na program kompajliran bez OpenMP-a. Renderovaćemo i sa stablom izgrađenim na jednoj niti i sa stablom izgrađenim na više niti, kako bismo otkrili eventualni gubitak performansi renderovanja uzrokovan paralelnom izgradnjom.

7. Zaključak

Ovo je rad tehnološki najbliži našem projektu. Pokazuje da OpenMP zadaci mogu paralelizovati rekurzivnu izgradnju uz malo izmjena i dokumentuje gdje to prestaje raditi: ograničen rani paralelizam, opadajući dobici, efekti rasporeda memorije i razlike među kompajlerima. Obrazac zadataka, prag i disciplinu mjerenja preuzimamo kao nadogradnju V3, a sam SBVH izostavljamo.

Literatura

[1] Waldemarson, G., Doggett, M. (2025). Parallel Axis Split Tasks for Bounding Volume Construction with OpenMP. GRAPP/VISIGRAPP 2025, Vol. 1, str. 347–354. doi:10.5220/0013317100003912.

[CP1] Burić, D., Fazlić, A., Doglod, Z., Dželo, A. (2026). P16: BVH Ray Tracer, Istraživački izvještaj, Checkpoint 1. Univerzitet u Sarajevu, Elektrotehnički fakultet.